

Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 5

з дисципліни «Розробка мобільних
застосунків під Android»

Тема: «Дослідження роботи з вбудованими датчиками.»

Виконав:

студент групи ІА-31

Сакун Дмитро

Перевірив:

Орленко С.П

Київ 2026

Мета роботи: ознайомитись з можливостями вбудованих датчиків мобільних пристроїв та дослідити способи їх використання для збору та обробки даних.

ЗАВДАННЯ

БАЗОВЕ (10/20 балів). Написати програму під платформу Андроїд, яка має інтерфейс для виведення даних з обраного вбудованого датчика (тип обирається самостійно, можна відслідковувати зміни значень і з декількох датчиків).

ПОВНЕ (20/20). Функціональність базового додатку додатково розширюється обробкою отриманих даних та виведенням їх у відповідній формі.

Примітка: конкретного варіанту не передбачено, студент сам обирає завдання та вигляд програми. Приклади очікуваних робіт:

- «будівельний рівень» з виведенням лінії горизонту та куту нахилу;
- компас з ілюстрацією стрілки (циферблату з позначеними сторонами світу);
- крокомір (підрахунок кількості кроків);
- додаток для вимірювання перевантажень в авто (G-force meter);
- автоматичне регулювання яскравості та екрану в залежності від рівня освітлення, але ще б додати автозаглушення екрану при піднесенні до перешкоди (до вуха під час розмови або «в кишені»), щоб уникнути ненавмисних дотиків;
- барометр з прогнозом погоди (мова про опади – зміна атмосферного тиску, а, можливо, і вологості з температурою).

HARD TASK (не обов'язково). Оскільки конкретного варіанту немає, то і конкретного завдання модифікацій додатку теж немає, але слід додати або додаткову аналітику, або використовувати інші засоби для підвищення функціональності програми. На прикладі крокоміру можна додати аналітику «за день» або «за тренування»: відобразити зміни активності, спробувати обчислити пройдену відстань чи витрачені калорії, для візуалізації можна побудувати відповідні графіки. На прикладі «вимірювача перевантажень в авто» можна спробувати додатково визначити швидкість та пройдену відстань, спробувати відмальовувати карту пройденого маршруту, якщо рух кільцевий – можна вести відлік часу проходження кола, або окремих його ділянок. Для визначення швидкості/дистанції можна спробувати інтегрування або через визначення місцезнаходження на основі геолокації.

Хід роботи

Для реалізації завдання було створено додаток за допомогою сучасного фреймворку Jetpack Compose. Для зчитування даних використовується апаратний датчик `Sensor.TYPE_ACCELEROMETER`.

Посилання на повний вихідний код проєкту (GitHub):

<https://github.com/verom1/SensorLab5>

Лістинг основного коду (MainActivity.kt):

```
package com.example.sensorlab5

import android.content.Context
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import kotlin.math.roundToInt

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    LevelScreen(context = this)
                }
            }
        }
    }
}

@Composable
fun LevelScreen(context: Context) {
```

```

var sensorData by remember { mutableStateOf(Triple(0f, 0f, 0f)) }

DisposableEffect(Unit) {
    val sensorManager = context.getSystemService(Context.SENSOR_SERVICE) as
SensorManager
    val accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)

    val listener = object : SensorEventListener {
        override fun onSensorChanged(event: SensorEvent) {
            if (event.sensor.type == Sensor.TYPE_ACCELEROMETER) {
                sensorData = Triple(event.values[0], event.values[1],
event.values[2])
            }
        }
        override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
    }

    sensorManager.registerListener(listener, accelerometer,
SensorManager.SENSOR_DELAY_UI)

    onDispose {
        sensorManager.unregisterListener(listener)
    }
}

val (x, y, z) = sensorData

val maxGravity = 9.81f
val normalizedX = (x / maxGravity).coerceIn(-1f, 1f)
val normalizedY = (y / maxGravity).coerceIn(-1f, 1f)

Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(16.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center
) {
    Text(
        text = "Будівельний рівень",
        fontSize = 28.sp,
        fontWeight = FontWeight.Bold,
        color = MaterialTheme.colorScheme.primary
    )
    Spacer(modifier = Modifier.height(40.dp))

    Box(
        modifier = Modifier
            .size(250.dp)
            .background(Color(0xFFE0E0E0), shape = CircleShape),
        contentAlignment = Alignment.Center
    ) {
        Canvas(modifier = Modifier.fillMaxSize()) {
            val radius = size.width / 2
            val center = Offset(size.width / 2, size.height / 2)

            drawCircle(
                color = Color.DarkGray,
                radius = radius,
                style = Stroke(width = 4.dp.toPx())
            )
            drawCircle(

```

```

        color = Color.Red,
        radius = radius * 0.15f,
        style = Stroke(width = 2.dp.toPx())
    )

    val bubbleRadius = radius * 0.12f
    val bubbleOffset = Offset(
        x = center.x - (normalizedX * (radius - bubbleRadius)),
        y = center.y + (normalizedY * (radius - bubbleRadius))
    )

    val isLevel = Math.abs(x) < 0.5f && Math.abs(y) < 0.5f
    drawCircle(
        color = if (isLevel) Color(0xFF4CAF50) else Color(0xFF2196F3),
        radius = bubbleRadius,
        center = bubbleOffset
    )
}

Spacer(modifier = Modifier.height(40.dp))

Card(
    modifier = Modifier.fillMaxWidth(),
    shape = RoundedCornerShape(16.dp),
    elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
) {
    Column(
        modifier = Modifier.padding(20.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Сирі дані акселерометра (м/с²)", fontWeight = FontWeight.Bold)
        Spacer(modifier = Modifier.height(8.dp))
        Text("X (вліво/вправо): ${"%0.2f".format(x)}")
        Text("Y (вгору/вниз): ${"%0.2f".format(y)}")
        Text("Z (обличчям вгору): ${"%0.2f".format(z)}")

        Divider(modifier = Modifier.padding(vertical = 16.dp))

        Text("Обчислені кути нахилу:", fontWeight = FontWeight.Bold)
        val angleX = (normalizedX * 90).roundToInt()
        val angleY = (normalizedY * 90).roundToInt()
        Text("Нахил по осі X: $angleX°")
        Text("Нахил по осі Y: $angleY°")
    }
}
}

```

Результати роботи:

Програму було успішно скомпільовано у файл .apk та протестовано. На екрані відображаються сирі дані з акселерометра по осях X, Y, Z, а також обчислені кути нахилу. Графічний інтерфейс (рухома бульбашка) коректно реагує на зміну положення пристрою.



Рис. 1. Відображення інтерфейсу програми у вертикальному положенні пристрою (гравітація діє по осі Y).

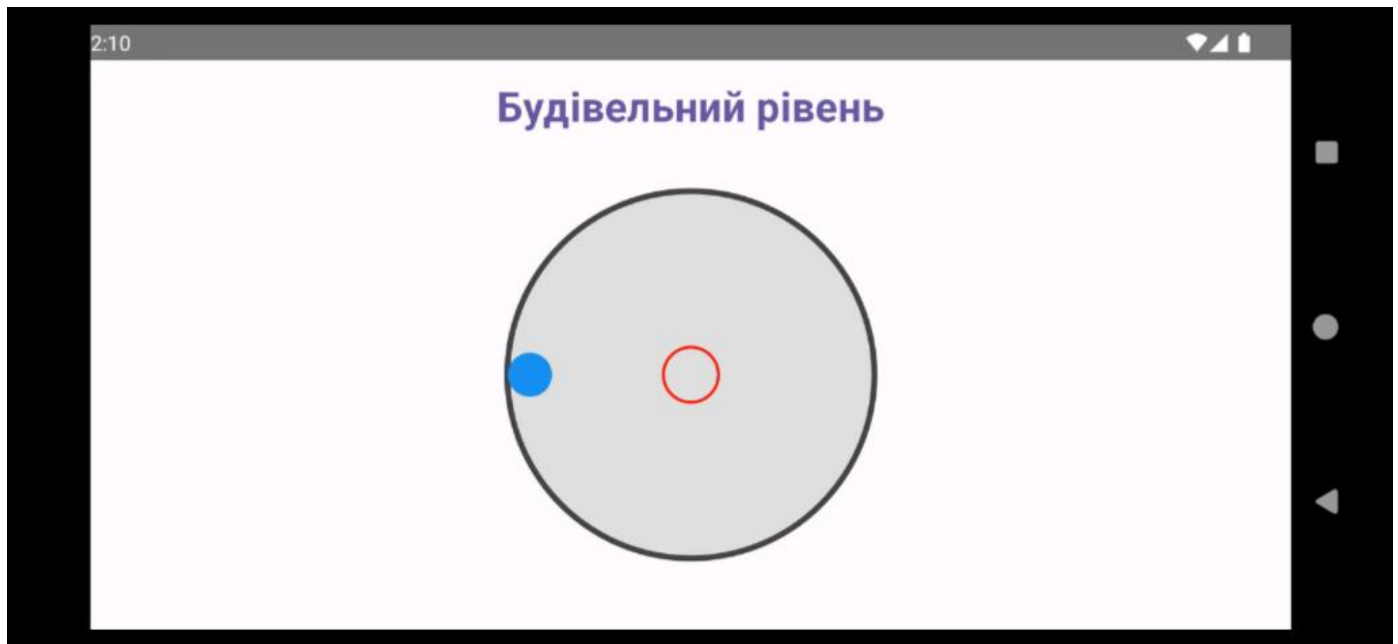


Рис. 2. Реакція програми на зміну положення пристрою (ландшафтний режим, гравітація діє по осі X).

Відповіді на контрольні запитання

1. Наведіть приклади вбудованих датчиків та величини які з них можна зчитати.

Сучасні смартфони мають апаратні та програмні датчики. Основні з них:

- **Акселерометр:** Вимірює прискорення пристрою (включно із силою тяжіння) вздовж трьох осей (X, Y, Z) у м/с^2 .
- **Гіроскоп:** Вимірює швидкість обертання пристрою в радіанах на секунду (rad/s).
- **Датчик освітленості:** Вимірює рівень навколишнього освітлення в люксах (lx).
- **Магнітометр:** Вимірює геомагнітне поле Землі в мікротеслах (μT).

2. Наведіть особливості роботи з вбудованими датчиками.

- **Керування життєвим циклом:** Датчики споживають багато енергії. Ресструвати слухача (`registerListener`) потрібно тільки тоді, коли екран активний, і обов'язково відписуватись при згортанні додатка.
- **Шум даних:** Апаратні датчики дуже чутливі. Для точних додатків дані потрібно пропускати через математичні фільтри (наприклад, фільтр низьких частот).
- **Фрагментація:** Не всі пристрої мають повний набір датчиків, тому перед початком роботи завжди потрібно перевіряти наявність датчика через `SensorManager`

Висновок:

Під час виконання лабораторної роботи я ознайомився з можливостями вбудованих датчиків мобільних пристроїв на базі ОС Android. Було успішно розроблено додаток «Будівельний рівень», який використовує клас `SensorManager` для отримання даних з акселерометра у реальному часі. Отримані дані було математично оброблено для переведення сирих значень (m/s^2) у кути нахилу (в градусах).

Для оптимізації роботи та економії заряду батареї було реалізовано правильне управління життєвим циклом датчика (відписка при неактивному екрані). Графічний інтерфейс був реалізований за допомогою Jetpack Compose, що дозволило динамічно візуалізувати дані на екрані. Усі завдання базового та повного рівня успішно виконані.